

Modul mahasiswa Lab 04 - Virtualisasi dan Container

Kebijakan kelas: Lab ini latihan formatif, tanpa tugas, nilai, atau penyerahan terpisah. Satu proyek besar dikerjakan oleh kelompok **3 orang**, dengan presentasi checkpoint minggu 7 (UTS) dan hasil akhir minggu 14 (UAS). Simpan hasil lab hanya bila berguna sebagai referensi atau bukti proses proyek. Baca [brief proyek kelompok](#). Bobot resmi tetap mengikuti RPS/LMS.

Cara membaca gambar: foto Docker Desktop dan Chrome adalah screenshot langsung dari praktik. Kartu terminal menunjukkan keluaran command yang benar-benar dijalankan dan ditata ulang agar terbaca. Setiap langkah bernomor memiliki gambar hasil; untuk laporan, ambil screenshot milik Anda sendiri setelah menjalankan perintah yang tertulis tepat di dekat gambar.

COMP6991031 | Praktikum kasus kerja harian | Windows PowerShell utama, Bash/Linux alternatif. Modul ini memuat **seluruh kunci challenge A-E**. Jalankan perintah dari root repo, kecuali bila langkah menyebut direktori lain. Setiap gambar adalah contoh hasil uji di komputer dosen; IP, waktu, ID, dan versi patch bisa berbeda. Simpan screenshot hasil praktikmu sendiri bila ingin menggunakannya sebagai referensi proyek kelompok.

Hasil belajar dan kasus

Di tim operasi, halaman status internal harus tersedia saat ada insiden. Kamu menerima tiga tugas: melihat kondisi layanan, memperbaiki halaman status tanpa membangun image baru, dan menyiapkan instance kedua ketika port utama sudah dipakai. Setelah lab, kamu dapat menjelaskan VM vs container, hubungan Docker client-daemon-registry, image vs container, namespaces/cgroups, port, bind mount, log, dan siklus `run -> stop -> start -> rm`.

Komponen	Arti pada kasus ini
Image <code>nginx:alpine</code>	Paket template server web yang tetap sama saat teks halaman berubah.
Container <code>cloudlab-site</code>	Proses Nginx yang berjalan dari image; dapat dihentikan dan dimulai ulang.
<code>site/index.html</code>	File di laptop yang dibaca Nginx melalui bind mount read-only.
<code>127.0.0.1:8088:80</code>	Port 8088 di laptop diteruskan ke port 80 dalam container, hanya pada loopback laptop.
Container <code>cloudlab-canary</code>	Instance kedua pada port host 8089 saat 8088 masih dipakai.

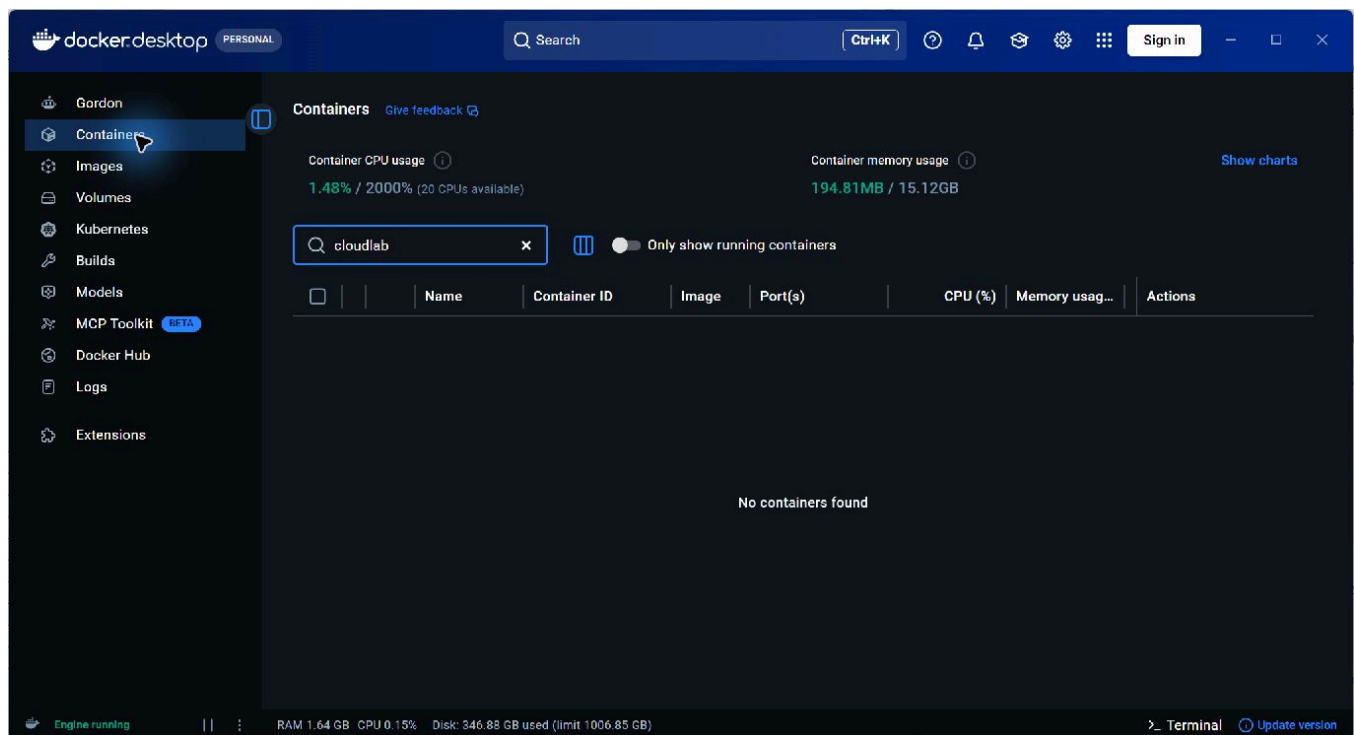
VM membawa guest OS sendiri di atas hypervisor. Container berbagi kernel host dan diisolasi dengan namespaces, sedangkan cgroups membatasi sumber daya. Docker CLI mengirim perintah ke daemon; daemon mengambil image dari registry lalu membuat container. Lab ini memakai container Linux melalui Docker Desktop/Engine, tanpa memasang hypervisor kedua.

0. Siapkan komputer dan repo

Pasang Docker Desktop/Engine dan Git. Pada Windows, buka Docker Desktop hingga status **Engine running**, lalu PowerShell dari root repo. Jika menggunakan Linux atau Codespaces, gunakan Bash. Jangan menyalin perintah PowerShell bind mount ke Bash karena sintaks path berbeda. Pastikan port 8088 dan 8089 tersedia; periksa `docker ps` sebelum memulai. Jika belajar dari repo template, buat repo pribadimu melalui **Use this template**, lalu clone atau buka Codespaces. Pada laptop dosen, folder ini sudah menjadi root proyek.

```
docker version
docker info --format '{{.ServerVersion}}'
docker ps
```

`docker version` menunjukkan CLI dan daemon dapat berbicara. `docker info --format` membatasi output ke versi server agar detail host tidak bocor ke laporan. `docker ps` membantu melihat apakah nama atau port lab sudah digunakan. Jika daemon belum aktif, buka Docker Desktop lalu ulangi.



Langkah: buka Docker Desktop dan jalankan `docker version`. Fungsi: memastikan Engine aktif sebelum menarik image. Cara kerja: CLI bertanya kepada daemon Docker; tanpa daemon, semua langkah berikutnya gagal. Baca hasil: indikator Engine running dan versi client/server tampil.

1. Kenali tiga image dan environment variable

```
docker pull nginx:alpine
docker pull python:3.12-alpine
docker pull node:22-alpine
docker image ls nginx
docker image ls python
docker image ls node
docker image inspect nginx:alpine --format '{{.Os}}/{{.Architecture}} {{.Id}}'
docker run --rm python:3.12-alpine python --version
docker run --rm node:22-alpine node --version
docker run --rm -e CLASS_SECTION=cloud python:3.12-alpine sh -c 'echo "$CLASS_SECTION"'
```

`pull` menyimpan image dari registry ke mesin. `image ls/inspect` membaca metadata image; ID akan tetap sama ketika hanya file `site/` yang berubah. `run --rm` membuat container sekali pakai dan menghapusnya saat selesai. Opsi `-e` menyuntik environment variable ke proses dalam container; baris terakhir harus mencetak `cloud`. Versi patch dan image ID bisa berubah sesuai tanggal pull.

A · image dan environment

```
PS> docker version --format 'Client / Server'
Client 29.6.2 | Server 29.6.2
PS> docker image ls nginx
nginx:alpine          df221db836e1          94.4MB          27.2MB    U
PS> docker image ls python
python:3.12-alpine    1b668429b351          83.6MB          21.3MB
PS> docker image ls node
node:22-alpine        0a7108bf6c7b          238MB           61.1MB
PS> docker run --rm python:3.12-alpine python --version
Python 3.12.15
PS> docker run --rm node:22-alpine node --version
v22.23.3
PS> docker run --rm -e CLASS_SECTION=cloud python:3.12-alpine sh -c 'echo "$CLASS_SECTION"'
cloud
```

Cuplikan output uji lokal aktual; baris/kolom dipilih dan ditata agar terbaca.

Langkah: jalankan tiga `docker pull`, `image ls`, `Python/Node versi`, dan contoh `-e CLASS_SECTION=cloud`. Fungsi: membedakan image tersimpan dari proses container sekali pakai. Cara kerja: proses `sh` dalam image `Python` membaca environment container lalu mencetak `cloud`; `--rm` membersihkan container setelah proses keluar. Baca hasil: ketiga image ada, versi muncul, dan output `cloud`. Gambar adalah cuplikan output uji lokal aktual yang ditata agar terbaca.

Sebelum membuat situs, jalankan `& .\tests\challenge.ps1` (PowerShell) atau `bash tests/challenge.sh` (dari Git Bash/Linux) sekali untuk melihat daftar tugas. **FAIL pada starter memang diharapkan**. Setelah tiga image tersedia, contoh lokal memberi 3 PASS, 11 FAIL; jumlah awal dapat berbeda bila image belum ditarik. Baca hint setelah FAIL sebagai urutan pekerjaan, lalu ulangi checker di akhir.

Challenge pada starter · petunjuk pekerjaan

```
PS> & .\tests\challenge.ps1
PASS A image nginx:alpine tersedia
PASS A image python:3.12-alpine tersedia
PASS A image node:22-alpine tersedia
FAIL B cloudlab-site berjalan dari nginx:alpine
FAIL B port 8088 hanya di 127.0.0.1
FAIL B folder site/ di-bind mount read-only
FAIL C halaman utama HTTP 200 di 8088
FAIL D halaman menampilkan INCIDENT-042: Pemeliharaan selesai
FAIL E cloudlab-canary berjalan dari nginx:alpine
Challenge: 3 PASS, 11 FAIL
```

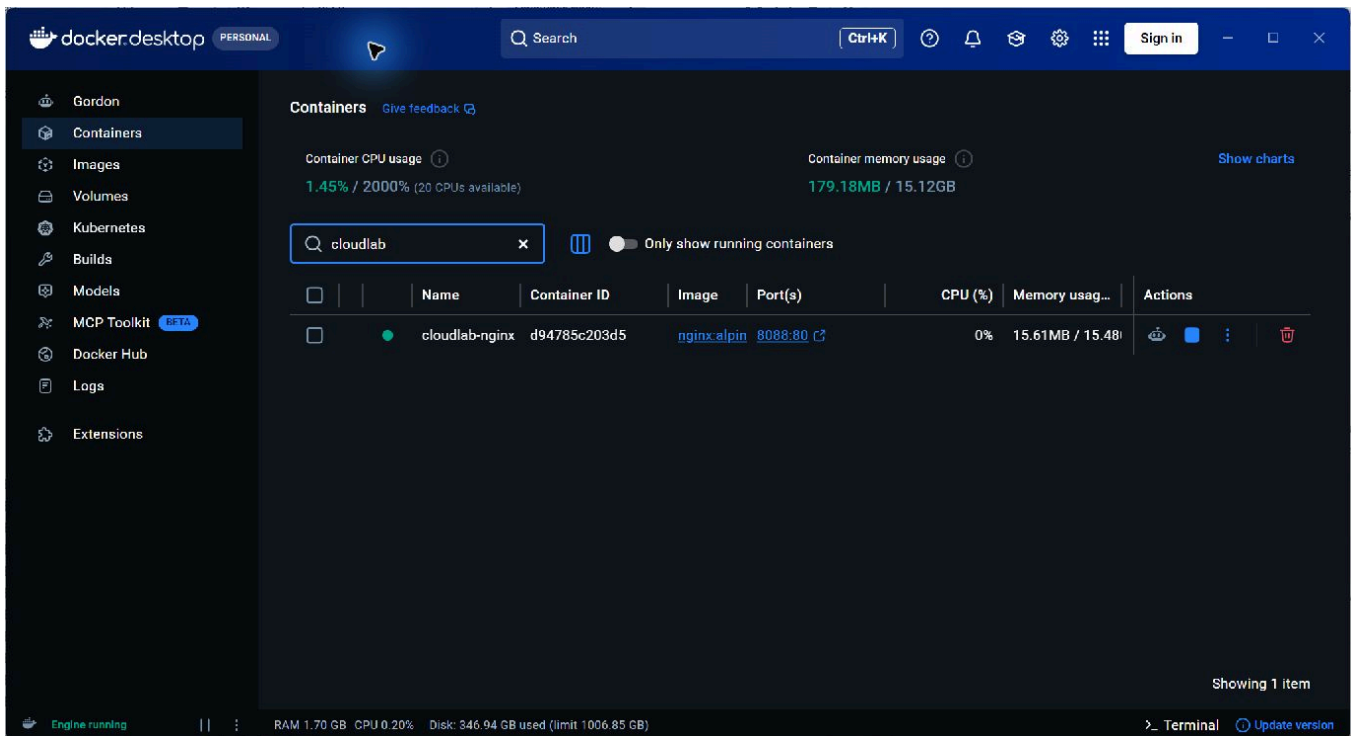
Cuplikan output uji lokal aktual; baris/kolom dipilih dan ditata agar terbaca.

Langkah: jalankan checker setelah menarik image tetapi sebelum deploy situs. Fungsi: memetakan syarat challenge A-E. Cara kerja: checker membaca image, metadata Docker, dan HTTP lokal tanpa membuat container. Baca hasil: A sudah PASS, B-E masih FAIL karena situs, hotfix, dan canary belum dikerjakan; ini kondisi awal yang benar. Gambar adalah cuplikan output uji lokal aktual yang ditata agar terbaca.

2. Operasikan container Nginx pertama

```
docker run --name clouddlab-nginx -d -p 127.0.0.1:8088:80 nginx:alpine
docker ps --filter name=clouddlab-nginx
curl.exe -I http://127.0.0.1:8088/
docker logs --tail 10 clouddlab-nginx
docker exec clouddlab-nginx sh -c 'hostname; ls /usr/share/nginx/html'
```

`run` membuat sekaligus memulai container. `-d` melepas proses dari terminal. `-p` meneruskan HTTP dari port laptop 8088 ke port container 80. `ps` menampilkan container hidup; `logs` membantu melihat request dan error; `exec` masuk untuk menjalankan perintah diagnostik tanpa mengubah image. `curl.exe -I` meminta header dan harus memberi HTTP 200. Buka juga <http://127.0.0.1:8088/> di Chrome: ini halaman bawaan Nginx, belum halaman kasus.



Langkah: jalankan `docker run`, `docker ps`, dan `curl.exe -I`. Fungsi: membuktikan proses web dan pemetaan port. Cara kerja: Docker mengikat `127.0.0.1:8088` ke `80/tcp` dalam container. Baca hasil: container Up, mapping `8088` ke `80`, dan HTTP 200.

Sekarang amati siklus container yang **sama**:

```
docker stop clouddlab-nginx
docker ps --filter name=clouddlab-nginx
docker ps -a --filter name=clouddlab-nginx
docker start clouddlab-nginx
docker ps --filter name=clouddlab-nginx
docker stop clouddlab-nginx
docker rm clouddlab-nginx
```

Setelah `stop`, `docker ps` tidak menampilkan container, tetapi `ps -a` menampilkan `Exited`. `start` memakai container lama dan mempertahankan identitasnya; `rm` menghapus objek container setelah berhenti. Image `nginx:alpine` tetap ada. Langkah akhir membebaskan port 8088 untuk situs kasus.

Siklus container • ID tetap saat stop/start

```
PS> docker stop cloudlab-nginx
PS> docker ps -a --filter name=cloudlab-nginx
d94785c203d5  nginx:alpine  Exited (0)  cloudlab-nginx
PS> docker start cloudlab-nginx
PS> docker ps --filter name=cloudlab-nginx
d94785c203d5  nginx:alpine  Up 127.0.0.1:8088->80/tcp
PS> docker stop cloudlab-nginx # kedua kali, sebelum rm
PS> docker rm cloudlab-nginx
```

Cuplikan output uji lokal aktual; baris/kolom dipilih dan ditata agar terbaca.

Langkah: `stop`, bandingkan `ps` dengan `ps -a`, lalu `start`. Fungsi: melatih diagnosis layanan berhenti. Cara kerja: Docker mengubah state proses container tanpa menghapus image atau membuat container baru. Baca hasil: ID sama, `Exited` sesudah stop dan `Up` sesudah start; `rm` dilakukan setelah stop kedua. Gambar adalah cuplikan output uji lokal aktual yang ditata agar terbaca.

3. Deploy halaman status dengan bind mount read-only

Pastikan file `site/index.html` berada di repo. Dari **root repo**, jalankan salah satu blok sesuai terminal. Jangan gunakan kedua blok sekaligus.

Windows PowerShell:

```
$siteDir = Join-Path (Get-Location).Path 'site'
$mount = "type=bind,source=$siteDir,target=/usr/share/nginx/html,readonly"
docker run --name cloudlab-site -d -p 127.0.0.1:8088:80 --mount $mount nginx:alpine
docker ps --filter name=cloudlab-site
docker port cloudlab-site 80
curl.exe -I http://127.0.0.1:8088/
```

Linux/Codespaces Bash:

```
site_dir="$(pwd)/site"
mount="type=bind,source=$site_dir,target=/usr/share/nginx/html,readonly"
docker run --name cloudlab-site -d -p 127.0.0.1:8088:80 --mount "$mount" nginx:alpine
docker ps --filter name=cloudlab-site
docker port cloudlab-site 80
curl -I http://127.0.0.1:8088/
```

Path `source` menunjuk folder **host**; `target` adalah folder yang dibaca Nginx **dalam container**. `readonly` membuat proses web tidak dapat mengubah file host. Buka <http://127.0.0.1:8088/> di Chrome. Di Codespaces, gunakan panel **Ports** untuk membuka 8088 setelah container hidup. Halaman awal harus menampilkan `STATUS_OK: Layanan normal`.

Status layanan internal

STATUS_OK: Layanan normal

Halaman ini disajikan Nginx dari folder `site/` pada komputer lokal.

Saat insiden diselesaikan, ubah kalimat status sesuai tiket kerja dan muat ulang halaman.

Langkah: jalankan `cloudlab-site` dengan `--mount`, lalu buka port 8088. Fungsi: menerbitkan halaman status internal. Cara kerja: Nginx membaca `site/index.html` langsung dari laptop; browser mencapai Nginx lewat mapping host ke container. Baca hasil: teks status awal dan HTTP 200.

Periksa mount dan port tanpa menebak:

```
docker inspect cloudlab-site --format '{{range .Mounts}}{{.Type}} {{.Destination}} RW={{.RW}}{{end}}'
docker port cloudlab-site 80
docker logs --tail 10 cloudlab-site
```

Cari `bind /usr/share/nginx/html RW=false` dan `127.0.0.1:8088`. `RW=false` adalah bukti mount read-only, bukan klaim dari nama folder. `logs` menunjukkan request yang baru dibuat browser/curl.

The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Gordon, Containers (selected), Images, Volumes, Kubernetes, Builds, Models, MCP Toolkit (beta), Docker Hub, Logs, and Extensions. The main panel is titled 'Containers' and shows system metrics: 'Container CPU usage' at 1.55% / 2000% (20 CPUs available) and 'Container memory usage' at 179.29MB / 15.12GB. A search bar contains 'cloudlab' and a toggle for 'Only show running containers' is turned on. Below this is a table of containers:

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Actions
☐	cloudlab-site	261e0e1108f8	nginx:alpin	8088:80	0%	15.72MB / 15.48	

At the bottom, it says 'Showing 1 item'. The footer bar shows 'Engine running', system resources (RAM 1.70 GB, CPU 0.25%, Disk 346.94 GB used / limit 1006.85 GB), and links to 'Terminal' and 'Update version'.

Langkah: periksa `docker inspect`, `docker port`, dan tampilan Containers di Docker Desktop. Fungsi: memastikan sumber file, mode mount, dan port benar. Cara kerja: `inspect` membaca metadata container yang sedang berjalan; `port` membaca aturan forwarding. Baca hasil: `RW=false`, container Up, dan host 8088 menuju port 80.

4. Challenge kasus kerja A-E

Kerjakan dahulu tanpa melihat kunci di bagian berikutnya bila ingin latihan mandiri. Simpan screenshot milikmu pada `hasil/bukti/lab04/`.

Bagian	Situasi dan tugas	Bukti yang diminta
A	Tim meminta bukti bahwa image nginx, Python, Node tersedia dan environment variable diterima proses sekali pakai.	<code>image ls</code> , versi, output <code>cloud</code> .
B	Situs internal harus hanya tersedia di laptop pada 8088; file dari host tidak boleh ditulis container.	<code>docker ps</code> , HTTP 200, <code>inspect</code> dengan <code>RW=false</code> .
C	Seseorang menghentikan <code>cloudlab-site</code> . Diagnosis dari <code>ps</code> , <code>ps -a</code> , dan log, pulihkan tanpa membuat image baru.	Kondisi gagal dan sesudah pulih.
D	Setelah insiden, ubah pengumuman menjadi <code>INCIDENT-042: Pemeliharaan selesai</code> tanpa rebuild.	Browser baru, <code>curl</code> , image ID sama, <code>RW=false</code> .
E	Port 8088 sudah dipakai situs utama. Buktikan bentrokan, lalu jalankan canary pada 8089 sambil situs utama tetap hidup.	Error port, dua port HTTP 200, dua container Up.

Kunci C - diagnosis dan pemulihan outage

```
docker stop cloudlab-site
curl.exe -I http://127.0.0.1:8088/
docker ps --filter name=cloudlab-site
docker ps -a --filter name=cloudlab-site
docker logs --tail 10 cloudlab-site
docker start cloudlab-site
curl.exe -I http://127.0.0.1:8088/
```

`curl.exe` pertama **seharusnya gagal** karena container berhenti; itu bukti insiden terkontrol, bukan kegagalan praktikum. `ps` kosong namun `ps -a` menunjukkan `Exited`; log memberi konteks request sebelumnya. `start` menghidupkan container yang sama. `curl.exe` terakhir harus HTTP 200. Dalam pekerjaan nyata, cek state dan log sebelum membangun ulang agar penyebab tidak tertutup.

C · diagnosis outage dan recovery

```
PS> docker stop cloudlab-site
PS> curl.exe -I http://127.0.0.1:8088/ # expected fail
curl.exe : curl: (7) Failed to connect to 127.0.0.1:8088 after 2033 ms: Could not connect to
server
[exit 7]
PS> docker ps -a --filter name=cloudlab-site
261e0e1108f8 nginx:alpine Exited (0) cloudlab-site
PS> docker start cloudlab-site
PS> curl.exe -I http://127.0.0.1:8088/
HTTP/1.1 200 OK
```

Cuplikan output uji lokal aktual; baris/kolom dipilih dan ditata agar terbaca.

Langkah: stop situs, uji HTTP gagal, baca `ps -a /log`, lalu start. Fungsi: meniru respons insiden layanan. Cara kerja: port 8088 berhenti menerima koneksi saat proses Nginx berhenti; `start` mengaktifkan kembali container yang sama. Baca hasil: `Exited` dan gagal koneksi berubah menjadi `Up` dan HTTP 200. Gambar adalah cuplikan output uji lokal aktual yang ditata agar terbaca.



This site can't be reached

127.0.0.1 refused to connect.

Try:

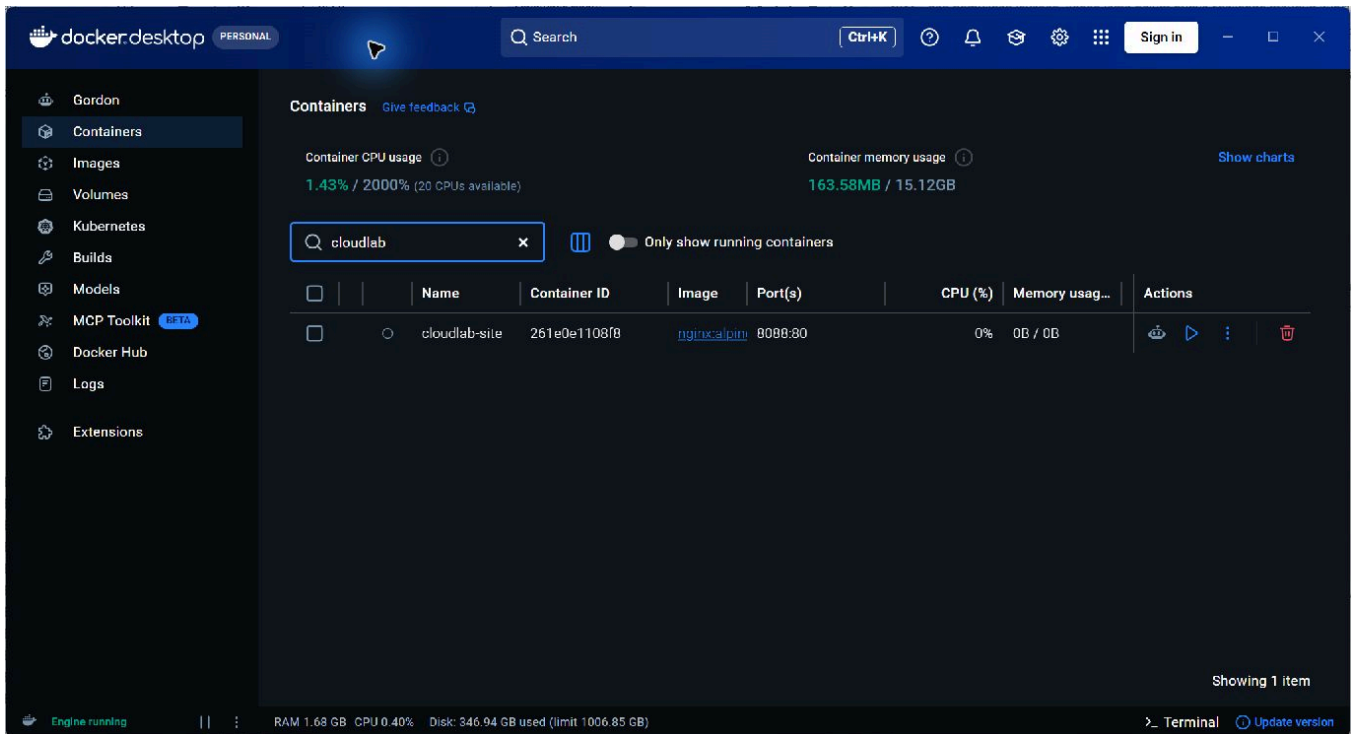
- Checking the connection
- [Checking the proxy and the firewall](#)

ERR_CONNECTION_REFUSED

Reload

Details

Langkah: setelah `docker stop cloudlab-site`, refresh `http://127.0.0.1:8088/` di Chrome. Fungsi: melihat gejala yang dialami pengguna. Cara kerja: tidak ada proses Nginx yang menerima koneksi host 8088. Baca hasil: `ERR_CONNECTION_REFUSED`; setelah `docker start`, refresh lagi dan halaman pulih.



Langkah: lihat `cloudlab-site` pada tab Containers setelah `docker stop`. Fungsi: membandingkan gejala browser dengan status container. Cara kerja: Docker Desktop membaca state dari daemon. Baca hasil: indikator container tidak aktif dan tombol Start tersedia; `docker ps -a` memberi status teks `Exited`.

Kunci D - hotfix konten tanpa rebuild

PowerShell - jalankan dari root repo. Perintah penulisan memakai UTF-8 tanpa BOM agar hasil HTTP bersih:

```
$imageBefore = docker image inspect nginx:alpine --format '{{.Id}}'
$sitePath = (Resolve-Path .\site\index.html).Path
$html = [System.IO.File]::ReadAllText($sitePath)
$html = $html.Replace('STATUS_OK: Layanan normal', 'INCIDENT-042: Pemeliharaan selesai')
[System.IO.File]::WriteAllText($sitePath, $html, (New-Object System.Text.UTF8Encoding($false)))
$imageAfter = docker image inspect nginx:alpine --format '{{.Id}}'
$imageBefore -eq $imageAfter
curl.exe -fsS http://127.0.0.1:8088/ | Select-String 'INCIDENT-042'
docker inspect cloudlab-site --format '{{range .Mounts}}{{.Destination}} RW={{.RW}}{{end}}'
```

Bash/Linux/Codespaces:

```
before=$(docker image inspect nginx:alpine --format '{{.Id}}')
sed -i 's/STATUS_OK: Layanan normal/INCIDENT-042: Pemeliharaan selesai/' site/index.html
after=$(docker image inspect nginx:alpine --format '{{.Id}}')
test "$before" = "$after" && echo 'image tetap sama'
curl -fsS http://127.0.0.1:8088/ | grep 'INCIDENT-042'
docker inspect cloudlab-site --format '{{range .Mounts}}{{.Destination}} RW={{.RW}}{{end}}'
```

Kamu juga boleh mengganti kalimat melalui editor VS Code lalu menyimpan file. Refresh Chrome. `True` pada PowerShell atau `image tetap sama` pada Bash berarti image tidak dibangun ulang; bind mount menyajikan file host terbaru. `RW=false` tetap menunjukkan Nginx tidak diberi hak tulis ke host.

Status layanan internal

INCIDENT-042: Pemeliharaan selesai

Halaman ini disajikan Nginx dari folder `site/` pada komputer lokal.

Saat insiden diselesaikan, ubah kalimat status sesuai tiket kerja dan muat ulang halaman.

Langkah: ubah satu kalimat di `site/index.html`, simpan, dan refresh Chrome. Fungsi: menyampaikan status insiden tanpa redeploy image. Cara kerja: bind mount membuat Nginx membaca perubahan file host pada request berikutnya. Baca hasil: INCIDENT-042: Pemeliharaan selesai muncul, sementara image ID sebelum/sesudah sama.

Kunci E - bentrokan port dan canary

Jalankan perintah pertama saat `cloudlab-site` **masih Up**. Kegagalan port yang muncul adalah hasil yang diharapkan:

```
docker run --name cloudlab-canary -d -p 127.0.0.1:8088:80 nginx:alpine
docker ps -a --filter name=cloudlab-canary
docker rm -f cloudlab-canary
docker run --name cloudlab-canary -d -p 127.0.0.1:8089:80 nginx:alpine
docker ps --filter name=cloudlab-
curl.exe -I http://127.0.0.1:8088/
curl.exe -I http://127.0.0.1:8089/
```

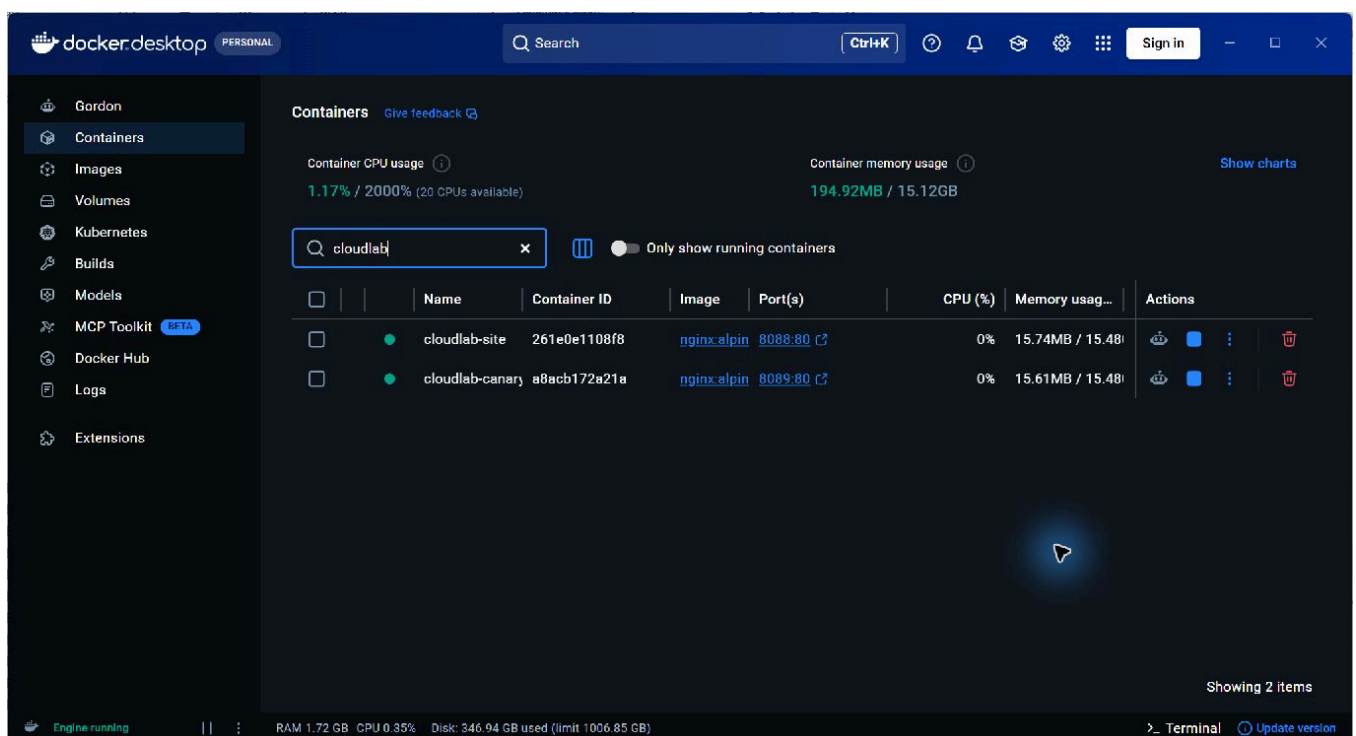
Jika Docker tidak membuat objek canary saat percobaan pertama, `docker rm -f cloudlab-canary` boleh menampilkan `No such container`; lanjutkan. Port host 8088 hanya dapat dimiliki satu proses, tetapi kedua container tetap bisa mendengar port **internal** 80 karena namespace jaringan mereka berbeda. Canary pada host 8089 adalah cara mencoba versi/instans kedua tanpa mematikan layanan utama.

```
E · port host bentrok, canary pindah ke 8089

PS> docker run cloudlab-canary -p 127.0.0.1:8088:80 # expected conflict
... Bind for 127.0.0.1:8088 failed: port is already allocated
[exit 125]
PS> docker ps -a --filter name=cloudlab-canary
cd32410c1537 cloudlab-canary Created
PS> docker rm -f cloudlab-canary
PS> docker run cloudlab-canary -p 127.0.0.1:8089:80
a8acb172a21a cloudlab-canary Up 127.0.0.1:8089->80/tcp
261e0e1108f8 cloudlab-site Up 127.0.0.1:8088->80/tcp
PS> curl.exe -I http://127.0.0.1:8088/
HTTP/1.1 200 OK
PS> curl.exe -I http://127.0.0.1:8089/
HTTP/1.1 200 OK
```

Cuplikan output uji lokal aktual; baris/kolom dipilih dan ditata agar terbaca.

Langkah: coba port 8088 untuk canary, baca error, bersihkan objek percobaan, lalu jalankan pada 8089. Fungsi: melatih diagnosis `port is already allocated`. Cara kerja: host tidak dapat mengikat 8088 dua kali; port container 80 tetap boleh sama. Baca hasil: kedua container Up dan HTTP 200 pada 8088 serta 8089. Gambar adalah cuplikan output uji lokal aktual yang ditata agar terbaca.



Langkah: lihat tab Containers sesudah menjalankan canary pada 8089. Fungsi: memeriksa bahwa situs utama tetap hidup saat instance kedua ditambah. Cara kerja: Docker Desktop menampilkan dua container dan dua pemetaan port host menuju port 80 masing-masing. Baca hasil: `cloudlab-site 8088:80` dan `cloudlab-canary 8089:80` sama-sama aktif.

Kunci A dan B - pemeriksaan akhir

Untuk A, gunakan perintah pada bagian 1. Untuk B, gunakan `docker ps`, `docker port`, `docker inspect`, dan `curl` pada bagian 3. Keduanya tidak membutuhkan kode aplikasi baru. Urutan A-E yang lengkap diuji oleh skrip berikut dari **root repo** setelah hotfix dan canary hidup:

PowerShell:

```
& .\tests\challenge.ps1
```

Bash/Linux/Codespaces/Git Bash:

```
bash tests/challenge.sh
```

Tes memeriksa hasil akhir yang dapat diamati: target **14 PASS, 0 FAIL**. Bagian environment variable pada A, riwayat stop/start pada C, dan percobaan bentrokan 8088 pada E tetap dibuktikan lewat screenshot/laporanmu, sebab status akhir saja tidak dapat membuktikan kejadian tersebut pernah dilakukan. Pada Windows, jalankan skrip Bash dari **jendela Git Bash**, bukan mengetik `bash` dalam PowerShell yang mungkin membuka WSL.



```
PS> & .\tests\challenge.ps1
PASS A image nginx:alpine tersedia
PASS A image python:3.12-alpine tersedia
PASS A image node:22-alpine tersedia
PASS B cloudlab-site berjalan dari nginx:alpine
PASS B port 8088 hanya di 127.0.0.1
PASS B folder site/ di-bind mount read-only
PASS C halaman utama HTTP 200 di 8088
PASS D halaman menampilkan INCIDENT-042: Pemeliharaan selesai
PASS D status lama sudah diganti
PASS E cloudlab-canary berjalan dari nginx:alpine
PASS E port 8089 hanya di 127.0.0.1
PASS E canary HTTP 200 di 8089
PASS E cloudlab-site tidak membuka port host lain
PASS E cloudlab-canary tidak membuka port host lain

Challenge: 14 PASS, 0 FAIL

Cuplikan output uji lokal aktual; baris/kolom dipilih dan ditata agar terbaca.
```

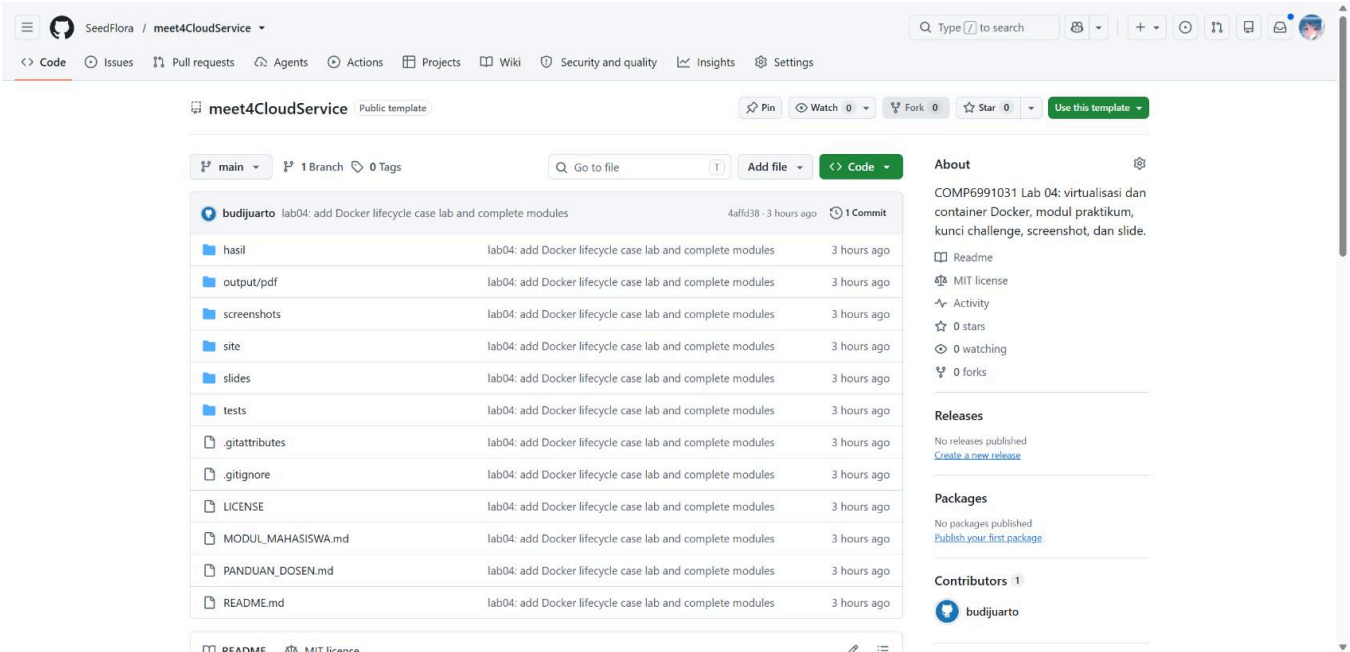
Langkah: jalankan `tests/challenge.ps1` atau `tests/challenge.sh`. Fungsi: mengecek image, situs, mount, HTTP, canary, dan port akhir. Cara kerja: skrip membaca Docker metadata dan melakukan request lokal; ia memberi PASS/FAIL tanpa memperbaiki konfigurasi. Baca hasil: `Challenge: 14 PASS, 0 FAIL`. Gambar adalah cuplikan output uji lokal aktual; simpan output praktikmu sendiri untuk laporan.

5. Laporan, Git, dan cleanup

Salin [templat laporan](#) ke `hasil/lab04.md`, isi dengan penjelasanmu, lalu masukkan screenshot pribadi ke `hasil/bukti/lab04/`. Jangan menyimpan output `docker info` penuh. Di repo pribadimu, jalankan dari root:

```
git status --short
git add site/index.html hasil/lab04.md hasil/bukti/lab04
git diff --cached --check
git diff --cached --name-only
git commit -m "lab04: pulihkan situs dan dokumentasikan insiden"
git push
```

`git status` menunjukkan perubahan, `git add` memilih hasil praktik, `diff --cached --check` menemukan whitespace yang bermasalah, dan `push` mengirim commit ke repo pribadimu. Jangan commit data host, kredensial, atau screenshot milik dosen sebagai bukti sendiri. Tidak perlu push image Docker ke registry. Setelah bukti tersimpan, bersihkan hanya container lab:



Langkah UI: sesudah `git push`, buka repo pribadi Anda di GitHub. Periksa branch `main`, pesan commit terbaru, serta file `hasil/lab04.md` dan screenshot pribadi. Foto menunjukkan repo template dosen sebagai contoh lokasi commit; repo mahasiswa akan memiliki nama dan isi commit sendiri. Perintah `git diff --cached --check` dijalankan sebelum commit untuk menolak whitespace bermasalah.

```
docker rm -f cloudlab-canary cloudlab-site
docker ps -a --filter name=cloudlab-
```

Image tetap ada sehingga dapat dipakai lagi tanpa pull ulang. `rm` menghapus container, sedangkan file `site/index.html` tetap ada pada host dan di Git.

Checkpoint cleanup: `docker ps -a --filter name=cloudlab-` hanya menampilkan header tanpa baris container lab. Buka tab **Containers** di Docker Desktop untuk memastikan `cloudlab-site` dan `cloudlab-canary` sudah tidak ada. Gambar pada langkah 3 dan 4 memperlihatkan kondisi sebelum cleanup sehingga perbedaannya dapat dibandingkan.

Jika ada kendala

Gejala	Pemeriksaan dan perbaikan
Cannot connect to Docker daemon	Buka Docker Desktop hingga Engine running; ulangi <code>docker version</code> .
Port 8088/8089 sudah dipakai	<code>docker ps</code> untuk mencari container lab. Jangan menghentikan container lain tanpa tahu pemiliknya; gunakan port kosong dan sesuaikan laporan bila perlu.
Nama container sudah dipakai	<code>docker ps -a --filter name=cloudlab-</code> ; jika sisa percobaan milikmu, <code>docker rm -f</code> nama tersebut, lalu ulangi.
--mount gagal di path berspasi	Jalankan dari root repo, gunakan tanda kutip pada seluruh nilai <code>--mount</code> seperti contoh PowerShell/Bash.

Gejala	Pemeriksaan dan perbaikan
Halaman tidak berubah	Pastikan <code>site/index.html</code> tersimpan, mount mengarah ke folder repo yang benar, lalu refresh Chrome.
Challenge C/D/E gagal	Periksa <code>docker ps</code> , <code>docker port</code> , <code>docker inspect</code> , dan isi <code>site/index.html</code> ; baca hint dari skrip sebelum mengulang.
PowerShell menolak <code>.ps1</code>	Jalankan <code>bash tests/challenge.sh</code> dari Git Bash/WSL, atau minta dosen membantu dengan kebijakan skrip kampus; langkah Docker tetap dapat dilakukan di PowerShell.